

プログラミング第2 最終課題(共同作成)

共同作成者

62315265 野澤仁

62303141 大石康太郎

62306070 橘内大輝

1. 作業の分担

グループの作業はメンバーの野澤が、プログラムの実装に必要なクラスや機能を整理し、要件定義を行った。そしてこの要件定義に従い、全体の工程を、データベース、プロセッサ、GUIの3つの層に分けた。データベース層ではデータベースを利用して会員情報や商品情報などの記録・管理を行い、プロセッサ層では入力に応じてそれらの情報を参照・変更する。また、GUI層では入出力の結果を画面上に視覚的に表示する。これら3つの層について主な担当者を、データベース層が野澤、プロセッサ層が大石、GUI層が橘内として分担した。そして自身の担当層が片付いた者から、他の層の手伝いや、本稿の執筆といった作業を進めた。なお、3人の作業はGitHubを用いて管理・共有して行った。

2. 実装したプログラム

2-1. データベース

データベース層では、会員情報、商品情報、貸出商品情報、予約商品情報の記録・管理を扱う。それぞれの情報はAccountクラス、Productクラス、RentalProductクラス、ReservedProductクラスの4つのクラスに分けた。尚、RentalProductクラスとReservedProductクラスはProductクラスのサブクラスとして定義したため、Productクラスの持つインスタンス変数とゲッターを全て継承している。各クラスのインスタンス変数を表2-1-1に示す。

表 2-1-1：各クラスのインスタンス変数

	情報	変数名
Accountクラス (会員情報)	ユーザーID	int userID
	ユーザーネーム	String usrName
	メールアドレス	String emai
	パスワード	String password
	クレジットカード番号	String cardNumber
	セキュリティ番号	String securityCodeStr
Productクラス (商品情報)	商品ID	int productID
	商品名	String productName
	商品説明	String description
	総在庫数	int totalStock
	現在在庫数	int currentStock
	予約件数	int numReservation
	貸出料金	int rentalFee
	画像名	String imageURL
	貸出期間	int rentalPeriod
	最短貸出可能日	LocalDate earliestRentalStart
RentalProductクラス (貸出商品情報)	貸出商品ID	int rentalProductID
	貸出ユーザーID	int userID
	貸出開始日	LocalDate rentalStart
	貸出期限日	LocalDate rentalDeadline
ReservedProductクラス (予約商品情報)	予約商品ID	int reservedProductID
	予約ユーザーID	int userID

表 2-1-1 の情報を記録するために本プログラムでは SQLite を用いたデータベースを利用した。SQLite の選定理由は、大学の Linux 環境で使用が可能であり、かつファイルベースのデータベースであるため、管理が容易であり、ユーザーが SQLite をインストールしていなくてもシステムで使用可能なためである。本システムで利用したデータベースについて構造を次の表 2-1-2 に示す。

表 2 - 1 - 2 : データベース構造

	データ名	型	主キー
CUSTOMER_INFOS	USER_ID	INTEGER	○
	USER_NAME	TEXT	
	EMAIL	TEXT	
	PASSWORD	TEXT	
	CARD_NUM	TEXT	
	SEC_CODE	TEXT	
PRODUCTS	PRODUCT_ID	INTEGER	○
	PRODUCT_NAME	TEXT	
	DESCRIPTION	TEXT	
	TOTAL_STOCK	INTEGER	
	CURRENT_STOCK	INTEGER	
	NUM_RESERVATION	INTEGER	
	RENTAL_FEE	INTEGER	
	IMAGE_URL	TEXT	
	RENTAL_PERIOD	INTEGER	
	EARLIEST_RENTAL_START	INTEGER	
RENTAL_PRODUCTS	RENTAL_PRODUCT_ID	INTEGER	○
	USER_ID	INTEGER	
	PRODUCT_ID	INTEGER	
	RENTAL_FEE	INTEGER	
	RENTAL_START	TEXT	
	RENTAL_DEADLINE	TEXT	
RESERVED_PRODUCTS	RESERVED_PRODUCT_ID	INTEGER	○
	USER_ID	INTEGER	
	PRODUCT_ID	INTEGER	
	EARLIEST_RENTAL_START	TEXT	

CUSTOMER_INFO テーブルと PRODUCTS テーブルの主キーは他のテーブルに共有され、レンタル中や予約中の商品の管理に使用している。尚、PRODUCTS テーブルは RegisterProducts.java を使用して csv ファイルからフィールドの値を取得して一括で初期化した。

実装の都合上複数のテーブルに対して同時に処理する必要があったことから、処理の途中でエラーが発生した場合でもデータの整合性が保てるよう、Connection クラスの setAutoCommit メソッドや rollback メソッドを活用した。また、メールアドレスなどに任意の文字列が含まれていてもシステムが正しく機能するよう、PreparedStatement メソッドを用いて SQL 文を動的に生成するといった工夫も行った。さらに、表 2-1-3 に示すメソッドをもつ SQL クラスを作成し、必要に応じてデータベースの情報を管理できるようにした。各メソッドでは処理に失敗した場合には、表 2-1-4 に示す例外を、次のプロセッサ層にメッセージ付きで投げるように実装した。

表 2 - 1 - 3 : SQL クラスのメソッドと機能

メソッド	機能
SQL()	SQLクラスのコンストラクタ。データベースと接続する。
void disconnect()	データベースとの接続を切る。
Account getCustomerInfo(String email)	メールをもとに会員情報を取得し、返す。
boolean addNewCustomer(String userName, String email, String password)	新しい会員情報を登録する。登録に成功したらtrue、失敗したらfalseを返す。
ArrayList<product> productQuery(Integer minPrice, Integer maxPrice, Boolean immediate)	価格や即時貸出可否の条件に応じて商品を検索し、該当商品のリストを返す。
boolean addNewProduct(Product product)	新しい商品をデータベースに登録する。登録に成功したらtrue、失敗したらfalseを返す。
boolean addNewRentalProduct(Product product, int userID)	商品のレンタル処理を実行する。処理に成功したらtrue、失敗したらfalseを返す。
boolean addNewReservedProduct(Product product, int userID)	商品の予約処理を実行する。処理に成功したらtrue、失敗したらfalseを返す。
ArrayList<RentalProduct> getRentalProducts(int userID)	ユーザーのレンタル中の商品を検索し、レンタル商品のリストを返す。
ArrayList<ReservedProduct> getReservedProducts(int userID)	ユーザーの予約中の商品を検索し、予約商品のリストを返す。
boolean returnProduct(RentalProduct rentalProduct, int userID)	商品の返却処理を実行する。処理に成功したらtrue、失敗したらfalseを返す。
boolean cancelProductReserve(Product product, int userID)	商品の予約キャンセル処理を実行する。処理に成功したらtrue、失敗したらfalseを返す。

表 2 - 1 - 4 : SQL クラスで投げる例外

例外	内容
DatabaseException	データベース関連のエラーが発生したことを示す。
NoResultFoundException	処理対象のデータがデータベースに見つからなかったことを示す。
UserAlreadyExistException	新規会員登録の際に、既にそのメールアドレスが登録されていることを示す。
RentalFailedException	レンタル処理で予期せぬエラーが発生したことを示す。
ReserveFailedException	予約処理で予期せぬエラーが発生したことを示す。
ReturnFailedException	返却処理で予期せぬエラーが発生したことを示す。
ReserveCancelFailedException	予約キャンセル処理で予期せぬエラーが発生したことを示す。

2 - 2. プロセッサー

プロセッサー層では、入力に応じてデータベースの情報を参照・変更する処理を行う。この処理のために、表 2-2-1 のメソッドをもつ Processor クラスを作成した。これらのメソッドは基本的に SQL クラスのメソッドを呼び出すことで処理を行っているが、try-catch 節を用いて SQL クラスで発生した例外を補足し、メッセージを付けて再び投げる設計になっている。また、この層ではメールアドレスなど入力された情報の形式を確認する処理も行っており、入力された情報の形式が正しくない場合にもメッセージ付きの例外を投げるように実装している。これにより Processor クラスからは、表 2-2-2 に示す例外が適切に投げられる仕組みになっている。

表 2 - 2 - 1 : Processor クラスのメソッド

メソッド	機能
Processor()	Processorクラスのコンストラクタ。SQLクラスのインスタンスを作成する。
boolean loginCheck(String email, String password)	入力されたメールアドレスとパスワードを検証し、ログイン処理をデータベース層に指示する。処理に成功したらtrue、失敗したらfalseを返す。
boolean newCustomer(String userName, String email, String password)	入力値を検証し、新しい会員情報の登録をデータベース層に指示する。登録に成功したらtrue、失敗したらfalseを返す。
ArrayList<Product> getProducts(Integer minPrice, Integer maxPrice, Boolean immediate)	価格や即時貸出可否の条件をデータベース層に渡し、検索結果の商品リストを返す。
boolean executeRental(Product product)	ログイン状態を確認し、データベース層にレンタル処理を指示する。処理に成功したらtrue、失敗したらfalseを返す。
boolean executeReserve(Product product)	ログイン状態を確認し、データベース層に予約処理を指示する。処理に成功したらtrue、失敗したらfalseを返す。
ArrayList<RentalProduct> getRentalProducts()	現在ログインしているユーザーのレンタル中の商品リストを取得するようデータベース層に指示し、取得したリストを返す。
ArrayList<ReservedProduct> getReservedProduct()	現在ログインしているユーザーの予約中の商品リストを取得するようデータベース層に指示し、取得したリストを返す。
boolean executeReturn(RentalProduct product)	ログイン状態を確認し、データベース層に返却処理を指示する。処理に成功したらtrue、失敗したらfalseを返す。
boolean executeReserveCancel(Product product)	ログイン状態を確認し、データベース層に予約キャンセル処理を指示する。処理に成功したらtrue、失敗したらfalseを返す。
Account getCurrentUser()	現在ログインしているユーザーのアカウントを返す。

表 2 - 2 - 2 : Processor クラスで投げる例外

例外	内容
DatabaseErrorException	データベース層に起因する技術的なエラーを示す。
EmptyInputException	必要事項が未入力であることを示す。
EmailPatternException	メールアドレスの形式が不適当であることを示す。
IncorrectInputException	入力内容が間違っていることを示す。
NoAccountException	登録されていないアカウントでログインしようとしていることを示す。
AlreadyRegisteredException	既に登録済みのメールアドレスで新規登録しようとしたことを示す。
NoProductFoundException	検索条件にヒットする商品が見つからないことを示す。
NotYetLoginException	ログインが必要な動作を未ログイン状態で行おうとしたことを示す。

2 - 3. GUI

GUI 層では入出力の結果を画面上に視覚的に表示する。この機能を果たすため、表 2-3-1 に示すメソッドとインナークラスをもつ GUI クラスを作成した。各メソッドでは基本的に Processor クラスのメソッドを呼び出すことによって処理を行っており、try-catch 節を用いて Processor クラスで発生した例外を補足し、エラーメッセージを表示するように実装している。

表 2 - 3 - 1 : GUI クラスのメソッドとインナークラス

メソッドとインナークラス	機能
GUI()	GUIクラスのコンストラクタ。Processorクラスのインスタンスを作成する。
void setGBCgrid(GridBagConstraints gbc, int x, int y, int width, int height)	GridBagLayout用の位置・サイズ指定を行う。
void createFrame()	アプリケーションのメインウィンドウを生成し、ウィンドウを閉じるときにログアウト処理とデータベースの切断を行う。
class ShowLoginPage extends MouseAdapter	ログインページのリンクがクリックされるとログインページへ遷移する。
void showLoginPage()	ログイン画面を作成し、ウィンドウに表示する。
class Login implements ActionListener	ログインボタンが押されるとログイン処理を実行する。ログインに成功すれば商品ページに遷移する。
class Logout implements ActionListener	ログアウトボタンが押されるとログアウトの処理を実行する。
void logout()	ログアウトの処理を実行する。
class ShowRegisterPage extends MouseAdapter	会員登録のリンクが押されると会員登録ページへ遷移する。
void showRegisterPage()	会員登録画面を構築し表示する。
class Register implements ActionListener	会員登録ボタンが押されると会員登録の処理を実行する。登録に成功すると完了メッセージを表示する。
class ShowProductPage implements ActionListener	商品一覧ページのリンクが押されると商品の一覧ページに遷移する。
class ShowProducts implements ActionListener	商品検索ボタンが押されると検索条件を取得して商品検索を実行する。その後検索結果をもとに商品一覧を更新する。
void showProducts(Integer minPrice, Integer maxPrice, Boolean immediate, JPanel resultPanel, JLabel errorLabel)	プロセッサ層に問い合わせで取得した商品リストをもとに、各商品の情報をもつパネルを動的に生成し、一覧ページに配置する。
class ShowProductDetail extends MouseAdapter	商品一覧パネルが押されると、商品の詳細画面を表示する。
void showProductDetail(Product product)	商品一覧から特定の商品パネルが押されると、商品の詳細情報を表示する詳細画面を構築する。
class Rental implements ActionListener	商品詳細画面でレンタルボタンが押されるとレンタル処理を実行する。
class Reserve implements ActionListener	商品詳細画面で予約ボタンが押されると予約処理を実行する。
class ShowRentalStatePage implements ActionListener	レンタル・予約商品の一覧ページのリンクが押されると、レンタル・予約商品ページへ遷移する。
void showRentalStatePage()	レンタル・予約商品一覧ページを構築する。左半分にレンタル商品一覧、右半分に予約商品一覧を表示する。
void showRentalProducts(JPanel rentalProductPanel)	プロセッサ層に問い合わせで取得したレンタル中の商品リストをもとに、レンタル商品の一覧を構築する。
void showReservedProducts(JPanel reservedProductPanel)	プロセッサ層に問い合わせで取得した予約中の商品リストをもとに、レンタル商品の一覧を構築する。
class ShowReturnDialog implements ActionListener	返却ボタンが押されると、最終確認画面を表示する。
class ReturnProduct implements ActionListener	返却時の最終確認で確認ボタンが押されると、返却処理を実行する。
class ShowReserveCancelDialog implements ActionListener	予約キャンセルボタンが押されると、最終確認画面を表示する。
class ReserveCancel implements ActionListener	予約キャンセルの最終確認で確認ボタンが押されると、予約キャンセル処理を実行する。

3. 使い方と機能

プログラム全体は GitHub にて公開している。(<https://github.com/bu-13a13ym-sitm/pro2-finalassignment/tree/release>)

プログラムは図 3-1 のように配置される必要がある。

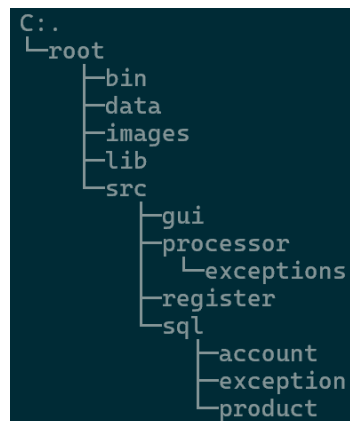


図 3 - 1：実行するためのプログラム配置

プログラムのコンパイル及び実行に際して、SQLite 用のドライバを取得し、lib 以下に jar ファイルを配置する必要がある。尚、ドライバのバージョンは大学の Linux にインストールされているバージョン 3.26.0 の SQLite に合わせて、3.26.0 以降直近のバージョンである 3.27.2 を使用した(取得元：<https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc/3.27.2>)。コンパイル及び実行は root ディレクトリにおいて行うことを想定している。コンパイルは Linux 環境では次のコマンドで実行することができる。

```
javac -d bin -cp "lib/*" src/*.java src/**/*.java src/**/*.java
```

Window 環境においてコンパイルを実行する際には、ワイルドカードが無効であるため、1 つ 1 つファイル名を root からの相対パスで正しく指定する必要がある。尚、GitHub 上ではコンパイル済みの class も併せて公開しているので、リモートリポジトリをクローンするか、zip ファイルをダウンロードしてローカルマシン上に展開することでコンパイルせずに実行することが可能である。プログラムの実行は次のコマンドで行う。

```
java -cp "lib/*:bin:bin/**/*.bin/**/*.bin" ClothesRentalSystem
```

コンパイル時と同様に、Window 環境において実行するにはワイルドカードを使用せずにクラスパスを指定する必要がある。また、クラスパスの並列にはコロンではなくセミコロンを使用する必要がある。

実行すると図 3-2 のようなウィンドウが表示され、メールアドレスとパスワードの入力が促される。ここに、正しいメールアドレスとパスワードを入力すればログインが完了し、レンタル商品の一覧ページに遷移する。間違ったメールアドレスやパスワードを入力した場合、図 3-3 にあるように「Email

adress or password is incorrect.」という表示がされ、正しいメールアドレスとパスワードの入力が再度求められる。

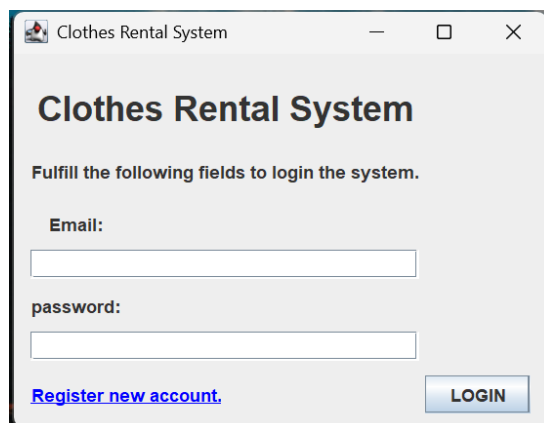
A screenshot of a web browser window titled "Clothes Rental System". The page has a light gray background. At the top, the title "Clothes Rental System" is displayed in a bold, dark font. Below the title, the text "Fulfill the following fields to login the system." is shown. There are two input fields: one labeled "Email:" and another labeled "password:". Below the "Email:" field, there is a blue hyperlink that says "Register new account.". To the right of the "password:" field, there is a blue button labeled "LOGIN".

図 3 - 2 : ログイン画面

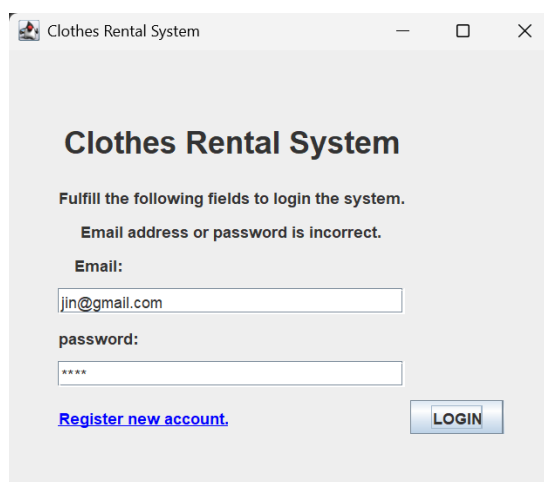
A screenshot of the same "Clothes Rental System" login page, but with an error message. The text "Email address or password is incorrect." is displayed above the input fields. The "Email:" field now contains the text "jin@gmail.com". The "password:" field contains four asterisks "****". The "Register new account." link and the "LOGIN" button are still present at the bottom.

図 3 - 3 : ログイン時に誤入力したときの表示

まだ会員登録をしていない場合、図 3-2 の左下にある「Register new account」をクリックすることで、図 3-4 の会員登録画面に移動することができる。この会員登録画面ではユーザーネームとメールアドレス、パスワード、クレジットカード番号、セキュリティコードの登録が求められる。また、このとき既に登録済みのメールアドレスを登録しようとする、図 3-5 のように「Your account is already registered.」と表示され、登録できないようになっている。登録が完了すると、図 3-6 のように登録が完了したことが知らされる。

Register new Account

Fulfill the following fields to register your account.

User Name:

Email:

password:

Enter your password again:

Credit Card Number:
 - - -

Security Code:

[Back to the login page.](#)

図 3 - 4 : 登録画面

Register new Account

Fulfill the following fields to register your account.

Your account is already registered.

User Name:

Email:

password:

Enter your password again:

Credit Card Number:
 - - -

Security Code:

[Back to the login page.](#)

図 3 - 5 : 登録済みのアカウントの入力時

Register new Account

Fulfill the following fields to register your account.

User Name:

Email:

password:

Enter your p

Credit Card Number:
 - - -

Security Code:

[Back to the login page.](#)

Register Completed

Register completed.

OK

図 3 - 6 : 登録完了時の表示

図 3-2 のログイン画面でログインができると図 3-7 の商品一覧ページに遷移する。この一覧ページでは商品の画像と商品名、料金、すぐに借りられるかどうかが一覧で表示され、これらをスクロールして見ることができる。また、下限価格や上限価格を入力することで、その価格帯の商品を検索することができ、右上の「Available Immediately」にチェックを入れるとすぐに借りられる商品のみに絞ることもできる。希望の条件の商品がない場合は図 3-7 のように「No product is found in the price range.」と表示される。アカウントを変えたい場合やログアウトをしたい場合には、左上の「Logout」のボタンを押すことで図 3-2 のログイン画面に戻ることができる。

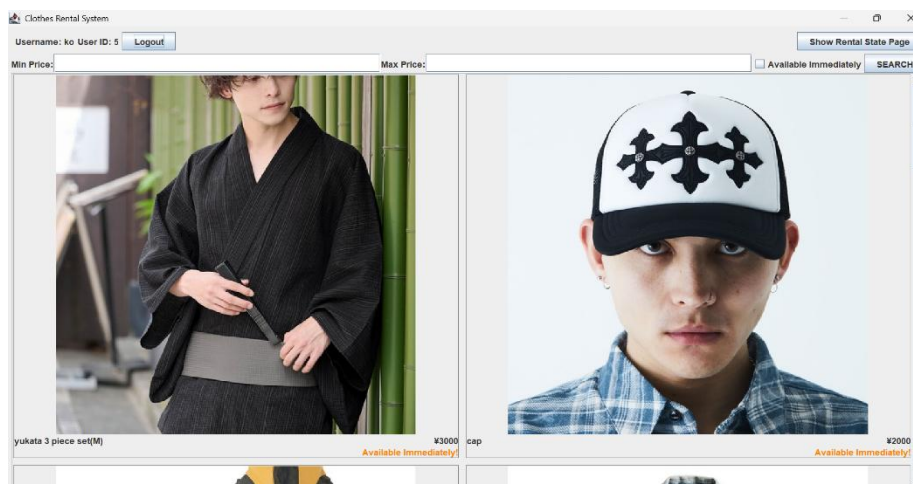


図 3 - 7 : 商品一覧ページ

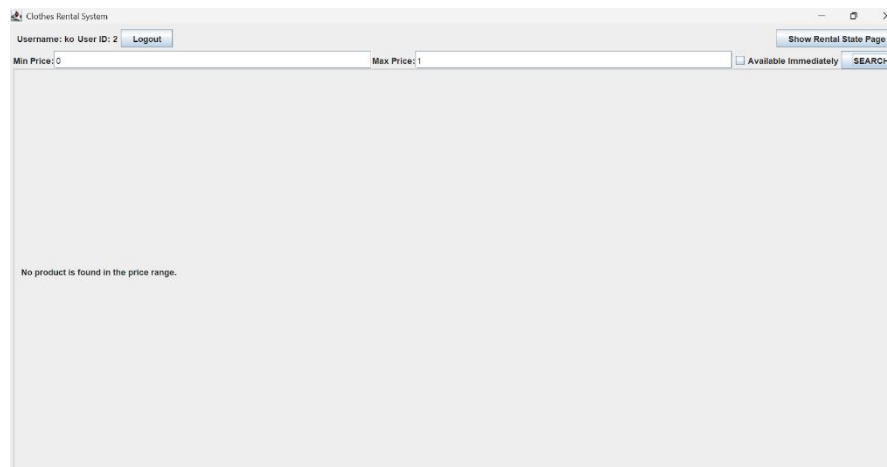


図 3 - 8 : 商品が見つからなかった場合の表示

図 3-7 の商品一覧ページの中から商品を選んでパネルをクリックすると、図 3-9 の小ウィンドウが現れ、商品詳細を見ることができる。そして、このウィンドウの右下の「Rental」ボタンをクリックし、図 3-10 の確認画面で「Yes」を選択すると、商品をレンタルすることができる。在庫がなくなった商品は図 3-11 の「yukata 3 piece set(M)」のように、一覧ページ上で次に借りられるようになる日付が表示される。予約をしたい場合には、この商品パネルをクリックして図 3-12 の商品詳細を表示し、右下の「RESERVE」をクリックすることで予約をすることができる。

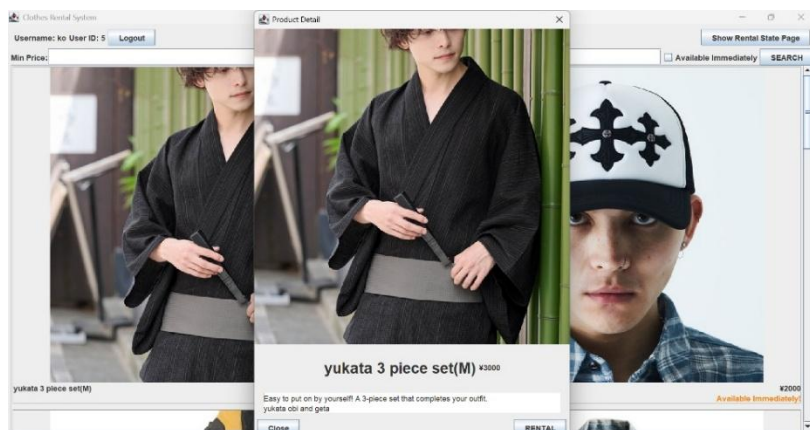


図 3 - 9 : 商品詳細ウィンドウ

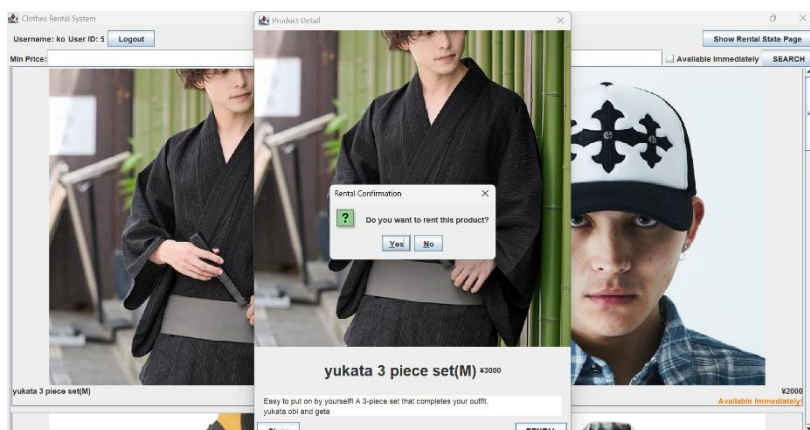


図 3 - 10 : 商品レンタルの確認画面

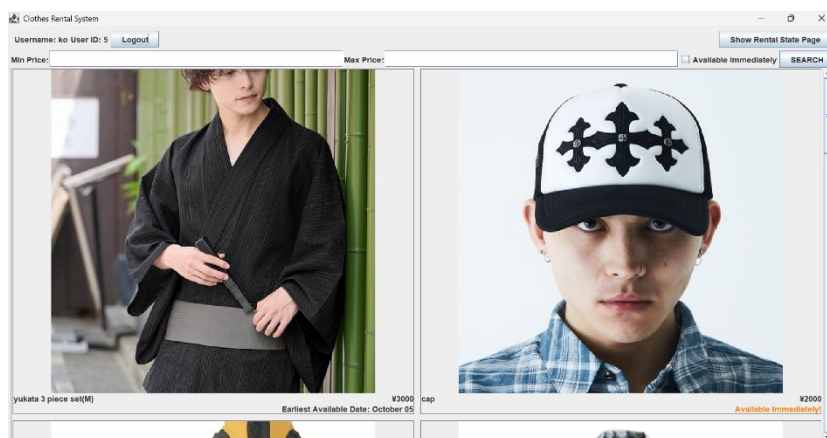


図 3 - 11 : 在庫切れ商品の表示



図 3 - 12：予約時の商品詳細画面

借りた商品や予約した商品は、図 3-7 の商品一覧ページの右上の「Show Rental State Page」をクリックすることで図 3-13 のページに遷移し、レンタル商品と予約商品を一覧で見ることができる。この画面で返したい商品を選び「RETURN」ボタンを押すと、図 3-14 の返却ウィンドウが現れ、さらにウィンドウの「Return」ボタンをクリックすることで商品を返却できる。また、図 3-13 のページの予約商品の「RESERVE CANCEL」ボタンをクリックすると、図 3-15 の予約キャンセルウィンドウが現れ、ウィンドウの「Cancel Reservation」ボタンをクリックすることで予約をキャンセルできる。そして再び商品ページに戻りたい場合は、右上の「Show Product Page」をクリックすることで、図 3-7 の商品一覧ページに戻ることができる。

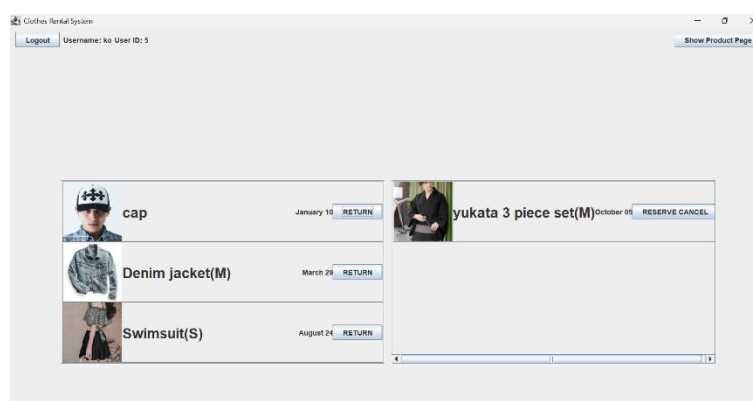


図 3 - 13：レンタル商品と予約商品の一覧ページ

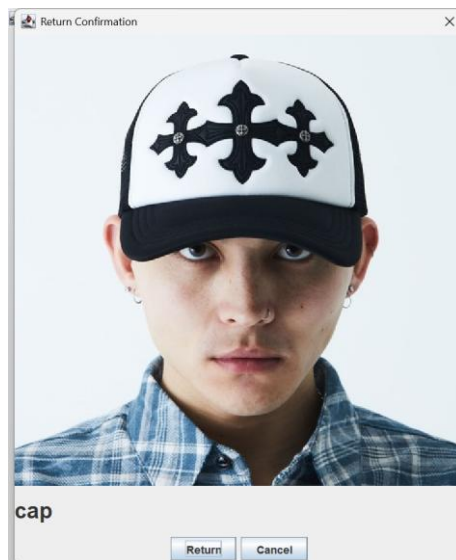


図 3 - 14：レンタル商品の返却ウィンドウ



図 3 - 15：予約商品の予約キャンセルウィンドウ

レンタル商品の中に返却期限遅れの商品がある場合、レンタル商品と予約商品の一覧ページで図 3-16 のように、正しい期限が赤く表示され期限から遅れていることが強調される。この商品の「RETURN」ボタンをクリックすると、図 3-17 の返却ウィンドウが現れ、延滞料が表示される。延滞料を確認し「Return」ボタンをクリックすることで商品を返却できる。

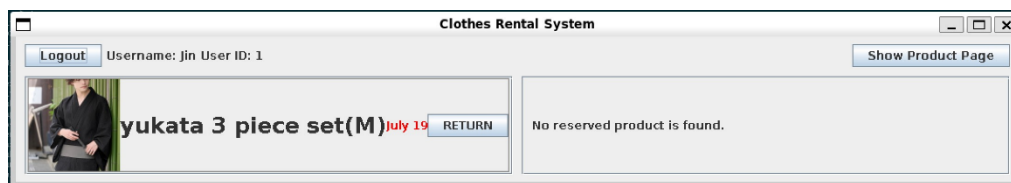


図 3 - 16：延滞している商品がある場合のレンタル商品と予約商品の一覧ページ

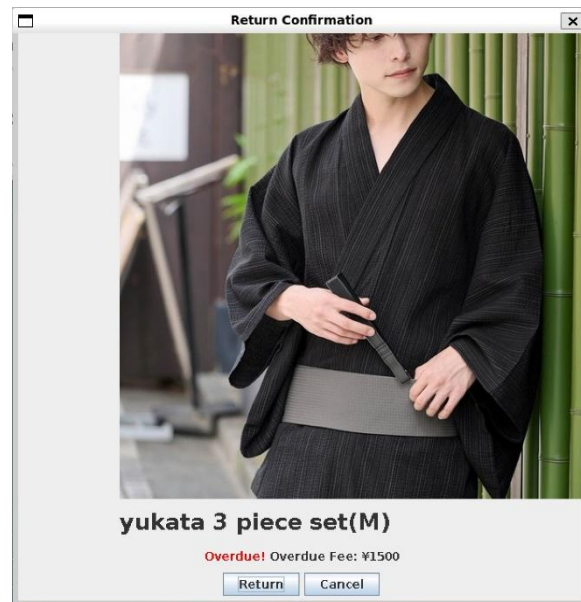


図 3 - 17：延滞商品の返却ウィンドウ

4. 共同開発をしてみたの感想

4 - 1. 野澤仁

以前 SIer で長期インターンシップに参加していた経験を活かしてチーム開発を進めることができた。その時はプロジェクトリーダーではなくメンバーとして開発に参加していたため、今回のように要件定義などをした経験はなかったが、プロジェクトリーダーの行動を思い出しながら、チーム開発において何が必要か改めて確認する機会となった。特に要件定義は、複数人で作業分担して開発を進めるうえで大変重要であったと感じた。関数のシグネチャを事前に定義しておくことで、担当間で必要なやりとりを極力減らすことができ、スムーズな開発につなげることができたと思う。また、プログラムを3つのパッケージに分離したこともチーム開発をスムーズに進めるうえで重要であったと考える。自分はインターンシップの参加経験を通してデータベースに関する知識を持っていたが、他のチームメンバーはデータベースに触れた経験がないため、データベース処理を完全に別のパッケージに分離することでブラックボックス化に成功し、短期間で開発を進めるにあたって、他のメンバーが1からデータベースに関する知識を身に付ける工程を省略することができた。開発が終了した今、今回の開発が他のメンバーたちにもデータベースに対して関心を抱き、学習する機会になることを願っている。

4 - 2. 大石康太郎

今回のプログラムの実装で実感したのは、複数人で一つのプログラムを実装する難しさだった。今回私が主に担当した箇所が中間層であったので、前後の層と上手く繋げるために他のメンバーが書いたコードを読んで理解する必要がある、当初はかなり苦労した。また、チームで GitHub を利用することに決めたものの、GitHub を用いたチーム開発は初めての経験だったので戸惑うことが多かった。間違ったブランチに修正を加えてしまい、他のメンバーの書いたプログラムとコンフリクトを起こしてしまったり、コード作成前にプルをし忘れて古いデータに変更を加えてしまったりと、様々なミスをしてしまった。しかし、チームでの開発を続けていく中で、次第に他のメンバーのコードを読むのも慣れ、また GitHub の使い方も少しずつ習得でき、操作に対する不安も減った。今回の実装を通じて、個人開発にはないチーム開発ならではの多くの困難を経験することができ、そしてそれらが乗り越えられる困難だと分かったことは、私にとって大きな学びになったように思う。今後の開発において複数人での開発の機会は必ずやってくるので、その時には今回の経験を活かし、より円滑な開発が行えるよう努めたいと思う。

4 - 3. 橋内大輝

初めての複数人でのプログラミングだったので、git に触れて扱い方を把握、実践できるという絶好の練習チャンスだと思い、git を用いた共同開発を行うこととなった。結果的にこの判断は非常に自らにためになる判断だったと思う。本課題に取り掛かる前は、git というシステムが存在し、将来どのような形であれコーディングを行うものとして git が扱えるのは必須のスキルであるということを知っている程度であったが、この感想を書くころにはある程度 git を用いた開発のフローを理解し、運用することができるようになった。また、普段の課題では 1 つのクラスまたはファイルで完結する問題がほとんどであったためこのように多数のクラス、ファイルを用いたプロジェクトは初めての体験であり、構想、設計、開発の流れについて理解が深まった。上記より本課題は非常にためになる体験だったと考える。一緒にやって難しかった点としては、やはり git merge を挙げたい。皆ある程度自分の作業が終わり、他の人の領域を手助けし始めた時間帯で複数人が同時に同じファイルをいじることが増えてきた。その際に 3 人で話して決めたブランチに自らの作業ブランチを merge するわけだが、初心者ゆえのコマンドの難解さやコンフリクト解消の手間など非常に苦戦する要素が多く、いまだスムーズにできるとはいいがたい。また、3 人が同じ時間帯にそろって開発ができるわけではないため、意思疎通が対面と比べてスムーズにいかないのも共同開発の難しさかなと感じた。

5. 参考にした Web ページ

IKURA TATSUO. SQLite 入門 - Let's プログラミング. JavaDrive. <https://www.javadrive.jp/sqlite/>, (閲覧日: 2025 年 7 月 20 日).

itoken1013. 【超入門】初心者のための Git と GitHub の使い方. 株式会社ラクス Tech Blog. <https://tech-blog.rakus.co.jp/entry/20200529/git>, (閲覧日: 2025 年 7 月 20 日).

nate. Java から SQLite に接続してデータベースの操作を行う方法. Biohac(k_e)r. <https://japbros-poco.main.jp/archives/7617>, (閲覧日: 2025 年 7 月 20 日).

@kuuuuumiiii. プルリクエストの一連の流れについて. Qiita.

<https://qiita.com/kuuuuumiiii/items/42d2d9ed11e3b29c22cf>, (閲覧日: 2025 年 7 月 20 日).